② Selection Bias

DATE / /

What is selection Bias?

→ when you select or analyze data in way that makes your conclusion misleading - often without realizing it.

Example:

↳ If you only survey people at a gym about their exercise habits, you'll conclude "Everyone exercise daily!" ⇒ But this ignores couch potatoes.

Key Types of Selection Bias

Type	What happen	Real-World Examples.
Data Snooping	keep searching data until you find a "pattern" (even if random)	finding a "winning" stock by testing 1,000 random strategies.
Cherry-Picking	Only show data that support your idea.	A politician citing only polls where they're ahead.
Survivorship Bias	Ignoring failure because they're not visible	"All ceos dropped out of college! (But you don't see the millions who failed)
Time Interval Bias	Picking a time range that makes trends look good.	"Our app grew 500%!" (After launching to just 10 users).

Why it Matters

- **False Discoveries**: You might think you found a real pattern, but it's just random Luck.
- **Bad Decision**: Business scientist, and politician make wrong moves based on biased data.

PAGE NO.....
DATE / /

How to Avoid it

1. Pre-Specify Hypotheses
   - Before looking at data, decide what you are testing (like a scientist designing an experiment).

2) Use Holdout Data
   - split your data
     - Training set (to explore)
     - Holdout set (to validate only once).

3) Random Sampling
   - Ensure your data represent the whole population (not just a convenient subset).

4) Target Shuffling (Permutation Test)
   - Shuffle your target variable and rerun the analysis. If the "pattern" still appears, it's probably fake.

Think like This!

- Coin Toss Contest: if 20,000 people flip coins, someone will get 10 heads in a row by chance. Don't assume they're magicians!
- Medical studies: if you test 100 drugs on the same data, one will "work" by luck. Always validate on new data.

Rules of Thumb:
- if you search too hard for "interesting" results, you'll find lies, not truth.

Conclusion

1) Strategies and holdout Data:

→ Fake strategies: You create 1000 random strategies (each with 100 daily returns.

→ Best strategy: You pick the best return for each day from all 1000 strategies to form a Frankenstein strategy.

2) Holdout Data:

→ Holdout Data: This is new, unseen data - 100 random returns for 100 days.

→ Why No strategies?: The holdout set contains only returns (no strategies) because you're testing how well your "best daily return" strategy works on future data.

dhawal dalal

PAGE NO.....
DATE / /

Code

```
# Detecting Fake Patterns (Data Snooping).

Import numpy as np.
# Simulate 1,000 "random" stock strategies (all actually useless)
np.random.seed(42)                    # 1,000 strategies over 100 days
fake_returns = np.random.normal(0, 1, (1000, 100))

# Data snooping: Pick the "best" strategy by max return.
best_strategy = fake_returns.max(axis=0).
print(f"Best strategy gained {best_strategy.mean():.2f}% daily (But it's pure luck!)")

# Holdout test: check if it works on new data.
holdout_returns = np.random.normal(0, 1, 100)
print(f"Holdout result: {holdout_returns.mean():.2f}% (Proves it was random)").
```

output

```
Best strategy gained 2.33% daily (But it's pure luck!)
Holdout result: -0.12% (Proves it was random)
```

Explanations

1) `fake_returns = np.random.normal(0, 1, (1000, 100))`.

This tells Numpy:

- → [0]: the mean of the normal distribution.
- → [1]: the std = how spread out the value are.
- → (1000, 100): make a 2D array (like a spreadsheet) with:
  - ↳ 1,000 rows → Each row = a fake strategy
  - ↳ 100 columns → Each column = a day's return.

PAGE NO.....
DATE / /

Imagine it like this:

Strategy↓/Day→	Day 1	Day 2	Day 3	.....	Day 100
Strategy 1	0.5	-1.1	0.3	...	0.2
Strategy 2	-0.4	0.2	1.3	...	-0.9
...					
Strategy 1000	0.1	0.7	-1.5	...	0.8

→ Each cell is random daily return from a normal distribution (mean = 0, std = 1).

→ So all strategies are just noise -- but some will look like they're working by accident.

In trading Note!
- Signal = actual useful information (e.g. a consistently profitable trading strategy).
- Noise = random, doesn't convey any meaningful signal

(2) `best_strategy = fake_returns.max(axis=0)`.

→ Returns the highest/max value per column (day).

Strategy↓/Day→	Day 1	Day 2	Day 3	...	Day 100
	0.5	0.7	1.3		0.9.

→ So, best_strategy is the winner of 100 values → one for each day (day's column).

→ best_strategy = [0.5, 0.7, 1.3, 0.9.]

→ what if `max(axis=1)`

Strategy 1 Strategy 2 // 100 ~~// 100~~

DATE / /

Imagine it like this!

Strategy↓/Day→	Day 1	Day 2	Day 3	.....	Day 100
Strategy 1	0.5	-1.1	0.3	...	0.2
Strategy 2	-0.4	0.2	1.3	...	-0.9
...					
Strategy 1000	0.1	0.7	-1.5	...	0.8

→ Each cell is random daily return from a normal distribution (mean = 0, std = 1).

→ So all strategies are just noise -- but some will look like they're working by accident.

In trading Note!
- Signal = actual useful information (e.g. a consistently profitable trading strategy).
- Noise = random, doesn't carry any meaningful signal

(2) `best_strategy = fake_returns.max(axis=0)`.

→ Returns the highest/max value per column (day).

Strategy↓/Day→	Day 1	Day 2	Day 3	...	Day 100
	0.5	0.7	1.3		0.9

→ So, best_strategy is the winner of 100 values → one for each day (day's column).

→ best_strategy = [0.5, 0.7, 1.3, 0.9]

→ what if `max(axis=1)`

Strategy 1	Strategy 2	// 100	~~100~~
0.5	1.3	0.7	

↳ Hence, for (max=1), best_strategy = [0.5, 1.3, 0.7]

PAGE NO.....
DATE / /

24) Regression to mean

1) What is Regression to the Mean?

↳ A statistical phenomenon where extreme results (very high or low) tend to move closer to the average when measured again.

Examples:

↳ A student who scores 100% on Test 1 (likely due to skill + luck) might score 85% on Test 2 (still great but less extreme).

↳ A baseball player with a record-breaking rookie season often perform slightly worse the next year.

Why?

→ Extreme outcomes are usually a mix of:

↳ Skill (consistent) + luck (random)

On the next try, the luck fades, pulling the result back toward the average.

2) Key Examples

Case	First Measurement (Extreme)	Second Measurement (Closer to Average)
Sports	Rookie of the Year	"Sophomore slump"
Testing	Top exam score	Slightly lower next te
Business	Best sales month	Normal sales next month.

3) Why This Matters.

↳ Misleading Conclusions: People often attribute the "decline" to something specific (e.g. "The rookie got lazy!") when it's just statistics.

↳ Selection Bias: [illegible]

26) Regression to mean

DATE / /

1) What is Regression to the Mean?

↳ A statistical phenomenon where extreme results (very high or low) tend to move closer to the average when measured again.

Examples:

↳ A student who scores 100% on Test 1 (likely due to skill + luck) might score 85% on Test 2 (still great but less extreme).

↳ A baseball player with a record-breaking rookie season often performs slightly worse the next year.

Why?

→ Extreme outcomes are usually a mix of:

↳ Skill (consistent) + luck (random)

On the next try, the luck fades, pulling the result back toward the average.

2) Key Examples

Case	First Measurement (Extreme)	Second Measurement (Closer to Average)
Sports	Rookie of the Year	"Sophomore slump"
Testing	Top exam score	Slightly lower next test
Business	Best sales month	Normal sales next month.

3) Why This Matters.

↳ Misleading Conclusions: People often attribute the "decline" to something specific (e.g. "The rookie got lazy!"), when it's just statistics.

↳ Selection Bias: If you only study extreme cases, you'll see this pattern everywhere.

PAGE NO.....
DATE / /

4. How to Avoid Mistaking it for Real change.

1. Compare to a Control Group:
   ↳ Did all top performer decline, or just the one you're focusing

2. Repeat Measurement
   ↳ Track over time to see if the trend holds.

3. Ask: Was Luck Involved?
   ↳ if yes, regression to the mean is likely.

Python code

2. (9) Regression to Mean

DATE / /

Explaination

```
import numpy as np
import pandas as pd.

# Simulate 1000 rookies : skill (50,80) + luck (-20 to +20)
(1) np.random.seed(42).
    skill = np.random.uniform (50, 80, 1000)
    rookie_luck = np.random.uniform (-20, 20, 1000)
    rookie_year = skill + rookie_luck

(2) # Top 10 "Rookies of the Year" (extreme highs)
    top_10 = np.argsort (rookie_year) [-10:]

(3) # Next year: same skill, new luck (regression to mean)
    sophomore_luck = np.random.uniform (-20, 20, 10).
    sophomore_year = skill [top_10] + sophomore_luck

# Compare
df = pd.DataFrame ({
        "Player": top_10,
        "Rookie Year": rookie_year [top_10]
        "Sophomore Year": sophomore_year,
        "Change": sophomore_year - rookie_year [top_10]
    3)
print (df.sort_values ("Rookie Year", ascending =
        False))
```

(1) Skill: A random skill between 50 and 80 is assigned to each rookie player. This is the base talent or ability of the player.

Luck: Luck is a random factor between -20 and +20, which represents good or bad fortune

PAGE NO.....
DATE / /

their can influence the player's performance.

- Rookie Year Performance :⇒ The rookie year performance is calculated by adding the skill and luck of each player. Some players will be lucky (high luck value), while others may be unluck (low luck value).

(2) # Top 10 "Rookies of the Year" (extreme highs).
top_10 = np.argsort(rookie_year)[-10:]
⇒ This sorts the rookie performance in ascending order, and the [-10:] extracts the indices of the top 10 players with the highest performance.

(3) Simulating the Sophomore Year (Regression to the Mean).

Sophomore_luck = np.random.uniform(-20, 20, 10).
Sophomore_year = skill[top_10] + sophomore_luck.

- Regression to the Mean : The key idea here is that in the rookie year, some players may have had exceptionally high luck. The next year (sophomore year), their luck is likely to "regress to the mean", meaning the luck will be closer to ~~the~~ average (not as extremes as in the rookie year).

- Sophomore luck : New luck values are drawn again from the range -20 to +20 for the top 10 rookies, which could either increase or decrease their performance.

- Sophomore Year Performance : The performance in the sophomore year is calculated using the same skill values from the rookie year. but with the new luck.

PAGE NO.....
DATE / /

(2) Sampling Distribution of a Statistics.

1.) What is a Sampling Distribution?

Imagine you:

↳ 1.) Take many small samples from a population (e.g. survey 100 people 1,000 times).

2.) Calculate a statistic (e.g. average income) for each sample

3.) Plot all these averages → This is the sampling distribution of the mean.

Key idea:

↳ It shows how your estimate (e.g. average) bounces around due to random sampling.

2.) Central Limit Theorem (why it Matters).

- Magic Rule: No matter how weird the original data, the sampling distribution of the mean becomes normal (bell-shaped) as samples grow.
- Example:
  - ↳ Original Data: Incomes are skewed (most people earn 50k, a few earn 5M).
  - ↳ Sampling Distribution of Mean Income. Bell shaped if you average 30+ people.

3) Standard Error (Precision Meter).

- Definitions: Standard deviation of the sampling distribution
- Translation: "How much does my sample average jump around if I repeat the study?

- formula:

$$SE = \frac{\text{Standard Deviation}}{\sqrt{n}}$$

- Bigger samples (n) → smaller SE → More precise estimate.

Key Takeaways

1. Sampling Distribution ≠ Data Distribution:
   - Data: Raw value (e.g., individual incomes → skewed).
   - Sampling Distribution: stats from sample (e.g. averages → normal)

2) Bigger Sample = Tighter Estimates
   - SE shrinks with $\sqrt{n}$ → Double precision, quadruple sample size.

3) CLT is Your Friend:
   - Even for weird data, average behave predictably with enough samples.

Real-World Analogy:
- Baking Cookies
  1) One cookie (Data Point): Might be too salty or sweet.
- Average of 30 cookies (Sampling Distribution):
  1) Almost always perfectly balanced.